TEM_UW_APP

Technical description

Lehigh TEM Felzer Lab

November 2026

Contents

1 Overview **2**

2 What files are what **2**

3 Setup **2**

4 Running a case **2**

5 Configuration **3**

6 Input data **3**

7 Documentation **3**

8 Limits **4**

9 Global input data **4**
9.1 Sources 4
9.2 Ozone 5
9.3 Nitrogen deposition 5
9.4 Land cover 5
9.5 References 6

10 Converting old files **6**
10.1 Control and parameter files 6
10.2 ASCII inputs 7

1 Overview

Prepares and runs a case of TEM_2026. The app regrids gridded NetCDF data to the model grid, writes the NetCDF inputs and `tem_config.yaml` of the model, and runs or submits `xtem`. It also builds the global input files and converts the files of the older TEM versions. The earlier CSV/XML workflow is not supported; its last version is the commit before this one.

2 What files are what

Path	Content
`case_submit.sh`	entry point, run from the case directory
`config/run_config.yaml`	all keys with defaults and comments
`src/driver.py`, `tem_climate.py`, `tem_config.py`	case builder: regridding, units, `tem_config.yaml`
`src/make_global_inputs.py`	builds `co2.nc`, `static.nc`, `lulc.nc` and the fire files for any region
`src/xml2yaml.py`, `src/ascii2nc.py`	convert old XML, parameter and ASCII files
`launchers/`	local and SLURM launchers
`examples/west/`	western United States example
`docs/`	`global_data.md` (sources of the input data), `converting.md`
`tools/make_docs.py`	builds the two PDFs

3 Setup

```
conda env create -f env.yaml && conda activate tem_uw_app
git clone https://github.com/Lehigh-TEM-Felzer-Lab/TEM_2026
make -C TEM_2026 -j4                  # serial: build/xtem
make -C TEM_2026 mpi -j4              # MPI, for n_cpus > 1: build_mpi/xtem
export TUP_ROOT=$PWD
```

4 Running a case

1. Copy `examples/west/run_config.yaml` to an empty directory.
2. Set `tem_src` (the TEM_2026 checkout), the climate files, the box and the years. For `n_cpus > 1` set `tem_exe: "{tem_src}/build_mpi/xtem"`.
3. Run `$TUP_ROOT/case_submit.sh` from that directory. With `run_mode: slurm` it submits with sbatch; with `run_model: false` it only builds the case.

```
run_config.yaml   tem_config.yaml
INPUT/            climate.nc static.nc lulc.nc co2.nc (lightning.nc population.nc)
OUTPUT/           tem.nc, written by the model
logs/
```

5 Configuration

`config/run_config.yaml` lists every key; keys you do not set take its values and unknown keys are an error.

Key	Description
`tem_src`, `tem_exe`	TEM_2026 checkout and executable (default `{tem_src}/build/xtem`)
`netcdf_climate_in_file`, `netcdf_climate_in_frequency`	climate files (path, glob or list) and their time step (hourly, daily, monthly)
`netcdf_static_in_file`, `netcdf_lulc_in_file`, `netcdf_co2_in_file`	static, land-cover and CO2 files in the layout of TEM_2026 `docs/io_structure.md`
`netcdf_fire_in_file`, `netcdf_population_in_file`	lightning flashes and population density, needed when `veg_fire: 1`
`varmap`	`<model variable>: <variable in your files>`; the model name is used first, the map only when the file lacks it
`horizontal_resolution`, `lon_min` to `lat_max`, `regrid_method`	model grid and regridding (`box_mean`, `bilinear`, `nearest`)
`spinflag`, `veg_fire`, `xnfertflag`, ...	the options of `tem_config.yaml`, one key each
`cmntecd`, `vegecd`, ...	parameter files, by default `{tem_src}/params/biogeography/`
`clmoutvars`, `temoutvars`	variables written to `OUTPUT/tem.nc`
`run_mode`, `n_nodes`, `n_cpus`, `walltime`, `partition`, `mem`, `sbatch_options`	sbatch settings as in pyUFS; the cells are shared between the `n_cpus` MPI tasks

Climate units are read from the `units` attribute and converted. `tmin` and `tmax` can replace `tair` and `dtr`, and `dewpoint` or `huss` can replace `vpr`. Daily and hourly data are averaged to months.

6 Input data

`climate.nc` is made by the app from your climate files. The other inputs come from `src/make_global_inputs.py`:

```
A="--out INPUT --cache downloads --lon-min -125 --lon-max -110 --lat-min 32 --lat-max 49 --res 0.5"
python src/make_global_inputs.py $A co2 --start 2015 --end 2024
python src/make_global_inputs.py $A soil
python src/make_global_inputs.py $A elev
python src/make_global_inputs.py $A wind
python src/make_global_inputs.py $A static
python src/make_global_inputs.py $A lulc --luh2 states.nc --luh2-static staticData_quarterdeg.nc --year 2015
```

Ozone, N deposition and the fire files come from files you supply (`ozone`, `ndep` and `fire` subcommands). Sources, download links, units and the land-cover assumptions are in docs/global_data.md. Files of older TEM versions are converted as in docs/converting.md. Then set the `netcdf_*_in_file` keys of `run_config.yaml` to the files.

7 Documentation

File	Content
run_config_Nov2026.pdf	every key of `run_config.yaml` with default and meaning
TEM_UW_APP-Technical-Description.pdf	this README and the documents in `docs/`

`python tools/make_docs.py` rebuilds both (needs pandoc and xelatex).

8 Limits

- The app writes monthly forcing, so it runs `run_dt: 1`, `run_dt_units: month` and rejects other steps. Daily forcing is not generated.
- Tested on the western United States example only; the inputs the app writes for it are identical to those shipped with TEM_2026.
- The global land-cover builder is not validated against the lab's earlier inputs.

9 Global input data

`src/make_global_inputs.py` writes the input files of TEM_2026 for any box and resolution (run it from the root of TEM_UW_APP). The commands below were run for a 0.5 degree box over the western United States. Needs `pip install xarray netcdf4 pandas scipy rioxarray` and `wget`.

```
A="--out INPUT --cache downloads --lon-min -125 --lon-max -110 --lat-min 32 --lat-max 49 --res 0.5"
python src/make_global_inputs.py $A co2 --start 2015 --end 2024
python src/make_global_inputs.py $A fire --fire-file tem_biogeography/static/fire.nc
python src/make_global_inputs.py $A soil
python src/make_global_inputs.py $A elev
python src/make_global_inputs.py $A wind
python src/make_global_inputs.py $A ozone --file <ozone.nc> --var <name>
python src/make_global_inputs.py $A ndep --file <ndep.nc> --var <name> --scale <factor>
python src/make_global_inputs.py $A static
python src/make_global_inputs.py $A lulc --luh2 states.nc --luh2-static staticData_quarterdeg.nc --year 2015
```

`static` merges elevation, soil, wind, ozone and N deposition into `static.nc`. `climate.nc` is made by the app from ERA5, MACA, gridMET or CMIP output. The regridding is the mean of the source cells in a target cell, so the target should be coarser than the source.

9.1 Sources

File	Variable (unit)	Source
`co2.nc`	`co2` (ppmv)	NOAA GML Mauna Loa annual mean; SSP scenarios for later years (Meinshausen et al. 2020)
`flash.nc`	`lightning_flashes` (km-2 month-1)	LIS/OTD monthly climatology (Cecil et al. 2014), from `fire.nc` of TEM_Biogeography
`popdens.nc`	`population_density` (km-2)	GPWv4 (CIESIN 2018), from the same `fire.nc`
`static.nc`	`elev` (m)	ETOPO 2022, 60 arc-second ice surface (NOAA NCEI 2022)
	`sand`, `silt`, `clay` (percent)	SoilGrids 2.0 at 5 km (Poggio et al. 2021), layers 0-5, 5-15 and 15-30 cm weighted by thickness, g kg-1 divided by 10
	`wind` (m s-1, by month)	WorldClim 2.1 mean wind 1970-2000 (Fick and Hijmans 2017); the height is 2 m, not 10 m
	`ozone` (AOT40, by month)	Gridded surface ozone metrics v2.7 (Evans and Sofen 2016; Sofen et al. 2016)
	`ndep` (mg N m-2 yr-1)	ISIMIP3b nitrogen deposition (Yang et al. 2026)
`lulc.nc`	cohorts of each cell	LUH2 v2h (Hurtt et al. 2020); potential vegetation of Ramankutty and Foley (1999)

Downloads (the script fetches all of these except the LUH2 files):

```
wget https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_annmean_mlo.txt
S=https://files.isric.org/soilgrids/latest/data_aggregated/5000m
wget $S/sand/sand_0-5cm_mean_5000.tif      # also silt, clay; 5-15 and 15-30 cm
N=https://www.ngdc.noaa.gov/thredds/fileServer/global/ETOPO2022/60s/60s_surface_elev_netcdf
wget $N/ETOPO_2022_v1_60s_N90W180_surface.nc
```

```
wget https://geodata.ucdavis.edu/climate/worldclim/2_1/base/wc2.1_10m_wind.zip
wget https://sage-public-files.s3.amazonaws.com/global-potential-vegetation/potveg_nc.tar.gz
wget https://luh.umd.edu/LUH2/LUH2_v2h/states.nc                    # 5.8 GB
wget https://luh.umd.edu/LUH2/LUH2_v2h/staticData_quarterdeg.nc
```

The CO2, soil, elevation, wind and fire steps were tested. The LUH2 step was tested on a synthetic file, not on the downloaded `states.nc`. Scenario CO2 after the observed years is not downloaded; pass a file with one `year ppmv` pair per line to `co2 --scenario` (see https://esgf-node.llnl.gov/projects/input4mips/). The lightning and population layers come from the LIS/OTD monthly climatology (https://www.earthdata.nasa.gov/data/catalog/ghrc-daac-lohrmc-2.3.2015) and GPWv4 (https://doi.org/10.7927/H49C6VHW, login).

9.2 Ozone

1. Create a CEDA account (https://help.ceda.ac.uk/article/39-ceda-account) and an access token (https://services.ceda.ac.uk/account/token/; valid for 3 days).
2. Download a monthly file near your grid resolution from the dataset record https://catalogue.ceda.ac.uk/uuid/c0455ab814224a05ac74642d3d44b73e/ (archive directory http://data.ceda.ac.uk/badc/proc-earth-model/data/ozone-metrics/v2.7):

```
D=https://dap.ceda.ac.uk/badc/proc-earth-model/data/ozone-metrics/v2.7
wget --header "Authorization: Bearer $CEDA_TOKEN" -O ozone.nc
 ↪  "$D/CCMI_SurfaceOzoneObs_1x1_Monthly_v2.7.nc?download=1"
ncdump -h ozone.nc | grep -i aot40      # variable name for --var
```

The variable names, the unit of AOT40 and the time axis of that file were not checked, because it needs a login. The script averages a time axis into a 12-month climatology; check the result. The data come from monitoring stations, so cells without stations are missing and TEM skips cells with missing data.

9.3 Nitrogen deposition

1. Open https://doi.org/10.48364/ISIMIP.600567.1 (ISIMIP3b simulated historical nitrogen deposition, v1.1). The file list holds the NHx and NOy files. The site blocks scripted requests: download in a browser, or use `pip install isimip-client` (https://github.com/ISI-MIP/isimip-client).
2. Total deposition is NHx plus NOy: `cdo add nhx.nc noy.nc ndep.nc`.
3. `python src/make_global_inputs.py $A ndep --file ndep.nc --var <name> --scale 3.1536e13`. The factor converts kg m-2 s-1 to mg N m-2 yr-1 (1e6 mg per kg times 3.1536e7 s per year). Check the unit in the file, and whether it is per kg N or per kg of compound.

Another source is the ACCMIP multi-model deposition of Lamarque et al. (2013) (https://doi.org/10.5194/acp-13-7997-2013).

9.4 Land cover

The script writes up to three cohorts per cell from the LUH2 fractions of one year and the cell area:

Cohort	LUH2 states	`currentveg`	`agstate`	`standage`
1 natural	`primf`, `primn`, `secdf`, `secdn`, `range`	potential vegetation of the cell	0	3000
2 crops	`c3ann`, `c4ann`, `c3per`, `c4per`, `c3nfx`	50 (Crops)	1	1
3 pasture	`pastr`	51 (Pasture)	1	1

`chrtarea` is the area in km2, `potveg` the TEM type of the potential vegetation, `subtype` equals `currentveg` and `FRI` is 2000. A cohort exists where its area exceeds 0.001 km2. The model counts the cohorts of each cell from the cells where `standage` has data. `cohort`, `potveg`, `currentveg` and `agstate` carry `flag_values` and `flag_meanings` that list the TEM vegetation types by name. Urban land is not simulated.

The 15 classes of Ramankutty and Foley map to TEM types (`params/biogeography/tcomm50_lulc.yaml` of TEM_2026) as 1 to 16, 2 to 18, 3 to 33, 4 to 9, 5 to 10, 6 and 7 to 4, 8 to 8, 9 to 14, 10 to 13, 11 and 12 to 15, 13 to 3, 14 to 21 and 15 to 2. This crosswalk, the `subtype` rule, `standage` of the managed cohorts and

`agstate` of pasture are assumptions of the script; the model developers should confirm them before a production run. Management layers (`tillflag`, `fertflag`, `irrgflag`) and land-cover change in time are not written, so a run uses `tlulcflag: 0`.

`lulc.nc` has `cohort` as dimension and as int64 coordinate; `lat` and `lon` are float32, categorical fields int64 with `_FillValue -9999`, other fields float32. The layouts of all files are in `docs/io_structure.md` of TEM_2026.

9.5 References

- Cecil, D. J., Buechler, D. E. and Blakeslee, R. J. (2014) Atmos. Res. 135-136, 404-414. https://doi.org/10.1016/j.atmosres.2012.06.028
- CIESIN (2018) Gridded Population of the World, Version 4 (GPWv4): Population Density, Revision 11. https://doi.org/10.7927/H49C6VHW
- Evans, M. J. and Sofen, E. D. (2016) Gridded Global Surface Ozone Metrics data (1971-2015), version 2.7. CEDA. https://doi.org/10.5285/897a3958-5bfc-4311-9bb0-01134bf6aefa
- Fick, S. E. and Hijmans, R. J. (2017) Int. J. Climatol. 37, 4302-4315. https://doi.org/10.1002/joc.5086
- Hurtt, G. C. et al. (2020) Geosci. Model Dev. 13, 5425-5464. https://doi.org/10.5194/gmd-13-5425-2020
- Lamarque, J.-F. et al. (2013) Atmos. Chem. Phys. 13, 7997-8018. https://doi.org/10.5194/acp-13-7997-2013
- Meinshausen, M. et al. (2020) Geosci. Model Dev. 13, 3571-3605. https://doi.org/10.5194/gmd-13-3571-2020
- NOAA NCEI (2022) ETOPO 2022 15 Arc-Second Global Relief Model. https://doi.org/10.25921/fd45-gt74
- Poggio, L. et al. (2021) SOIL 7, 217-240. https://doi.org/10.5194/soil-7-217-2021
- Ramankutty, N. and Foley, J. A. (1999) Global Biogeochem. Cycles 13, 997-1027. https://doi.org/10.1029/1999GB900046
- Sofen, E. D. et al. (2016) Earth Syst. Sci. Data 8, 41-59. https://doi.org/10.5194/essd-8-41-2016
- Yang, J., Tian, H. and Büchner, M. (2026) ISIMIP3b Simulated Historical Nitrogen Deposition Input Data, v1.1. ISIMIP Repository. https://doi.org/10.48364/ISIMIP.600567.1

10 Converting old files

For runs set up with the older TEM versions (TEM_Monthly, TEM_Biogeography, GDSTEM and others).

10.1 Control and parameter files

```
python src/xml2yaml.py us_tem45_lulc.xml -o tem_config.yaml
python src/xml2yaml.py tcomm45_lulc.ecd -o tcomm50_lulc.yaml
```

`xml2yaml.py` turns an XML control file or an `.ecd`/`.dat` parameter file into YAML. Every tag becomes a key and the XML comments become YAML comments. It repairs the known typos of the old files (a mismatched closing tag, an unclosed quote) and lists the ASCII input names it finds, since TEM_2026 reads NetCDF through the `netcdf_*_in_file` keys. The other keys keep their old names; compare the result with `templates/tem_config.yaml` of TEM_2026.

10.2 ASCII inputs

```
python src/ascii2nc.py clm tair.csv prec.csv dtr.csv nirr.csv vpr.csv -o climate.nc
python src/ascii2nc.py clm wind.csv aot40.csv -o static.nc --grid climate.nc
python src/ascii2nc.py elev elev.csv -o static.nc --grid climate.nc
python src/ascii2nc.py soil soil.csv -o static.nc --grid climate.nc
python src/ascii2nc.py ndep ndep.csv -o static.nc --grid climate.nc
python src/ascii2nc.py co2 co2.csv -o co2.nc
python src/ascii2nc.py lulc lulc.csv -o lulc.nc --grid climate.nc
```

Repeated calls with the same `-o` add to the file. The kinds and the record layouts are listed at the top of `src/ascii2nc.py`:

Kind	Record
`clm`	lon, lat, VAR, area, year, total, max, ave, min, jan to dec, region (year -999: climatology by month)
`daily`	lon, lat, VAR, area, year, month, day, value, region
`elev`	lon, lat, ELEV, area, elev, region
`soil`	lon, lat, SOIL, area, sand, silt, clay, wsoil, source, region
`ndep`	lon, lat, year, value
`co2`	year, ppmv
`lulc`	lon, lat, VAR, year, cohort, srccohort, standage, chrtarea, potveg, currentveg, subtype, agstate, agprevstate, tillflag, fertflag, irrgflag, disturbflag, disturbmonth, FRI, slashpar, vconvert, prod10par, prod100par, vrespar, sconvert, region

Variables are renamed as TEM_2026 expects (`CLDINESS` to `clouds`, `DTAIRRNG` to `dtr`, `VPRPRESS` to `vpr`, `AOT40` to `ozone`). The model counts the cohorts from `lulc.nc`, so the old cohort-count file (`MXCOHRTS`) is not needed. The old files list only cells with data: `--grid` takes a NetCDF file of the full grid, usually the climate file, and restores the empty cells.

Checked by writing the example inputs of TEM_2026 as ASCII, converting them back (equal to the originals within the ASCII rounding) and running the model on the result (maximum GPP 423.98 against 424.01). The `daily` and `ndep` kinds were not tested. The old ASCII fire inputs and the old output files are not converted; `tem_utils` converts old HIST output.